

composing.c

Three Hat Tiles Sharing the Central Vertex $v[12]$

MyHatTileStuff/composing.c — Technical Reference

June 16, 2026

Purpose

`composing` builds a single static figure: three copies of the hat tile arranged as a *pinwheel* around one shared vertex. The base hat is placed at orientation 0° ; two further copies are rotated by 120° and 240° ; and all three are positioned so that vertex $v[12]$ of every copy lands on exactly the same lattice point. The result is rendered to `img/composing.svg`.

Unlike its larger sibling `grow_symmetric2` (which grows clusters ring-by-ring under the RL0 rulebook), `composing` does *no* rule-based growth. It is a focused geometric construction: rotate, attach, verify the shared vertex, draw. This makes it an ideal program for studying the attachment primitives and the C calling conventions they rely on.

Data Types

Four small structs from `core/cycle.h` drive the whole program. Their sizes matter for the discussion of pointer arguments in Section 5.

```
typedef struct { int v; int x; int y; } Coord;           /* a lattice point (12 bytes) */
typedef struct { int n; Coord v[MAX_VERTS]; } Cycle;    /* a polygon boundary (large) */
typedef struct { int cycle_count; Cycle cycles[MAX_CYCLES]; } Poly; /* >=1 cycles (very large) */
typedef struct { Coord a; Coord b; } Edge;             /* an oriented boundary edge */
```

A `Coord` is a tagged lattice point: v is the valence tag (6, 4 or 3) and (x, y) are integer coordinates. A `Cycle` holds up to `MAX_VERTS` = 384 vertices, so it is several kilobytes wide; a `Poly` holds up to `MAX_CYCLES` = 32 such cycles and is larger still. These size facts are exactly why the library passes `Cycle` and `Poly` *by pointer* while it passes `Coord` *by value*.

Program Walkthrough

Setup: load and rotate

The tile is read from `tiles/hat.tile` into a `Tile` struct. (The file header comment mentions `preferences/focus.tile`; the code actually loads `tiles/hat.tile`.)

Three orientations are produced. `rot[0]` is the base hat as loaded. `cycle_transform_lattice` applies the exact origin-centred 60° -step lattice rotation; $t = 2$ gives 120° and $t = 4$ gives 240° .

The shared centre is read off the *base* hat: `center = tile.base.v[12]`, namely $(3, 1, -1)$.

```
Tile tile;
tile_load("tiles/hat.tile", &tile);

Cycle rot[NUM_HATS];           /* NUM_HATS = 3 */
rot[0] = tile.base;
cycle_transform_lattice(&tile.base,
    &rot[1], tile.lattice, 2); /* 120 deg */
cycle_transform_lattice(&tile.base,
    &rot[2], tile.lattice, 4); /* 240 deg */

Coord center = tile.base.v[CENTER_VERTEX];
/* CENTER_VERTEX = 12 -> (3,1,-1) */
```

Seed: the first hat

The growing figure is tracked in two parallel arrays. `hats[k]` stores the placed hat *cycle* (for drawing); `poly[k]` stores the merged *polygon* after $k + 1$ hats have been glued together (for the next attachment). Hat 0 seeds both:

```
Poly poly[NUM_HATS];
Cycle hats[NUM_HATS];
hats[0] = rot[0];
poly[0].cycle_count = 1;
poly[0].cycles[0] = rot[0];
```

The Attachment Loop

This is the heart of the program: it glues hat 1 and then hat 2 onto the figure, insisting each time that the new copy's `v[12]` fall on the shared centre.

Outer loop — $k = 1, 2$. At the start of iteration k the figure `poly[k-1]` already contains k hats. We attach `rot[k]` to it. `build_boundary_edges` fills `edges[]` with the outline of `poly[k-1]` and returns the edge count `ec`.

Search loops — `be`, `te`. We try gluing the new tile along every *base* boundary edge `be` ($0 \dots ec$) against every *tile* edge `te` ($0 \dots \text{rot}[k].n$). Both loops carry the guard `&& !ok`, so the search stops the instant a valid placement is accepted.

Acceptance — **three tests**. A candidate placement must pass all of:

1. `try_attach_tile_poly_ex` returns non-zero — the tile fits along (be, te) without overlap, yielding the merged polygon `out` and the placed cycle `aligned`.
2. `out.cycle_count == 1` — the merge produced no holes.
3. `coord_eq(aligned.v[12], center)` — the new copy's vertex 12 sits on the shared centre.

Test 3 is what makes this a pinwheel rather than an arbitrary edge-to-edge join. On success we record `hats[k]` and `poly[k]` and set `ok = 1`; if no (be, te) works the program reports an error and exits.

```
Edge edges[512];
for (int k = 1; k < NUM_HATS; k++) {
    int ok = 0;
    int ec = build_boundary_edges(
        &poly[k-1], edges);
    for (int be = 0; be < ec && !ok; be++) {
        for (int te = 0;
             te < rot[k].n && !ok; te++) {
            Poly out;
            Cycle aligned;
            if (!try_attach_tile_poly_ex(
                &poly[k-1], &rot[k],
                tile.lattice, be, te,
                &out, &aligned))
                continue;
            if (out.cycle_count > 1) continue;
            if (!coord_eq(aligned.v[CENTER_VERTEX],
                          center)) continue;
            hats[k] = aligned; /* keep cycle */
            poly[k] = out;    /* keep merge */
            ok = 1;
        }
    }
    if (!ok) { /* cannot attach */ return 1; }
}
```

The `edges[512]` buffer is allocated once, outside the loop, and reused on every iteration: `build_boundary_edges` overwrites it each time and returns the current length, so stale entries past `ec` are simply never read.

The Ampersand (&): Pass-by-Pointer in C

The two calls inside the loop raise a natural question:

Why is there *no* `&` on `edges` in `build_boundary_edges(&poly[k-1], edges)`, but there *is* a `&` on `aligned` in `try_attach_tile_poly_ex(..., &aligned)`?

What & does

C passes every argument *by value* — the function receives a private copy. To let a function write back into a caller’s variable (an *output parameter*), or to avoid copying a large struct, you pass a *pointer* to it. The unary & (“address-of”) operator turns an object into a pointer to that object: if `aligned` has type `Cycle`, then `&aligned` has type `Cycle *`. The relevant signatures are:

```
int build_boundary_edges (const Poly *p, Edge *edges);
int try_attach_tile_poly_ex(const Poly *base, const Cycle *tile_variant,
                           int lattice, int base_edge_index, int tile_edge_index,
                           Poly *out, Cycle *aligned_out);
int coord_eq(Coord a, Coord b); /* note: by value, no pointer */
```

The key rule: arrays decay, scalars/structs do not

`aligned` **needs &**. It is declared `Cycle aligned`; — a *single struct object*. Its name has type `Cycle`, but the parameter `aligned_out` wants a `Cycle *`. A lone struct does not turn itself into a pointer, so you must apply & explicitly: `&aligned` is the `Cycle *` the function writes its result through. The same holds for `&out` (a `Poly`) and for `&poly[k-1]` (a `Poly` array *element* — one struct, hence &).

`edges` **needs no &**. It is declared `Edge edges[512]`; — an *array*. In C an array name used in an expression *decays* to a pointer to its first element: `edges` already has type `Edge *`, exactly what the parameter wants. Adding & would give `&edges` of type `Edge (*)[512]` (pointer-to-array-of-512) — a different, wrong type that the compiler would reject.

```
/* single struct objects -> take address */
Poly out;          /* &out      : Poly * */
Cycle aligned;     /* &aligned : Cycle * */
poly[k-1];         /* &poly[k-1] : Poly * */

/* array object -> already a pointer */
Edge edges[512];   /* edges      : Edge * */
                  /* &edges    : Edge (*)[512] WRONG */

build_boundary_edges(&poly[k-1], edges);
/*                  ^struct ^array */

try_attach_tile_poly_ex(&poly[k-1], &rot[k],
                       tile.lattice, be, te, &out, &aligned);
/*      ^struct ^elem      ^out ^struct */
```

Both are still “output parameters”

The distinction is purely about *type*, not intent. Both `edges` and `aligned` are written *by* the callee and read *by* the caller afterwards. `build_boundary_edges` fills `edges[0..ec-1]` and returns `ec`; `try_attach_tile_poly_ex` fills `*out` and `*aligned_out` and returns a success flag. The only reason one gets a & and the other does not is that an array hands you a pointer for free, while a single struct does not.

Contrast: passing by value

The acceptance test uses `coord_eq(aligned.v[12], center)` with *no* ampersands. `coord_eq` takes two `Coord` arguments *by value*: a `Coord` is only 12 bytes, so copying it is cheap and the function has no need to modify the caller’s data — it just reports equality. The general pattern across the codebase is therefore:

Object	How it is passed	At the call site
<code>Coord</code> (small, read-only)	by value	bare: <code>center</code>
<code>Cycle</code> / <code>Poly</code> (large, or written)	by pointer	<code>&aligned</code> , <code>&out</code>
<code>Edge edges[512]</code> (array)	decays to pointer	bare: <code>edges</code>

Argument-by-argument summary

Call site arg	Declared as	Parameter type	Why
<code>&poly[k-1]</code>	<code>Poly poly[3]</code> (element)	<code>const Poly *</code>	struct $\Rightarrow$ take address
<code>edges</code>	<code>Edge edges[512]</code>	<code>Edge *</code>	array decays, no &
<code>&rot[k]</code>	<code>Cycle rot[3]</code> (element)	<code>const Cycle *</code>	struct $\Rightarrow$ take address
<code>&out</code>	<code>Poly out</code>	<code>Poly *</code>	struct out-param $\Rightarrow$ &
<code>&aligned</code>	<code>Cycle aligned</code>	<code>Cycle *</code>	struct out-param $\Rightarrow$ &
<code>center</code>	<code>Coord center</code>	<code>Coord</code> (value)	small struct, by value

SVG Rendering

After the three hats are placed, `main` computes a bounding box over all `hats[k].v[i]`, picks a pixel scale $SC = 50$ and margin $M = 24$, and writes the SVG. Tetrille coordinates (v, x, y) are mapped to screen space by `svgx/svgy` (with $\sqrt{3}$ pre-computed in `g_r3`):

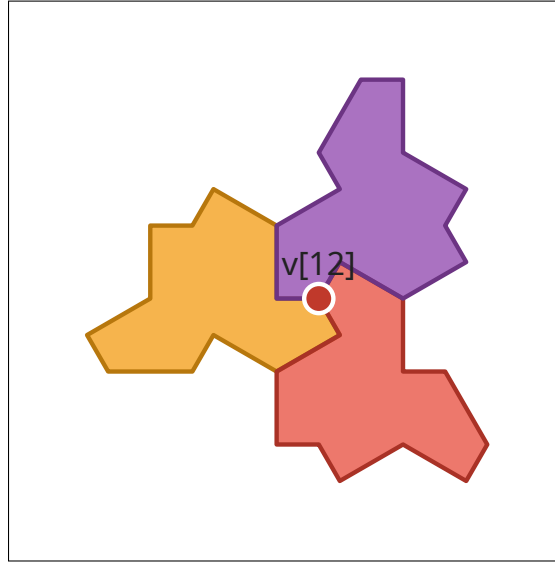
$$(x_s, y_s) = \begin{cases} \left(\frac{\sqrt{3}}{2}(x+y), \frac{1}{2}(y-x) \right) & v = 6 \\ \left(\frac{\sqrt{3}}{4}(x+y), \frac{1}{4}(y-x) \right) & v = 4 \\ \left(\frac{x+\frac{1}{2}y}{\sqrt{3}}, \frac{1}{2}y \right) & v = 3 \end{cases}$$

SVG's y -axis points down, so the draw routine emits $y_{\text{svg}} = o_y - SC \cdot y_s$. The three hats are filled in amber, red and purple; a small dark-red disc and a `v[12]` text label mark the shared centre.

Element	Colour	Hex
Hat 0 (base, 0°)	Amber	#f39c12
Hat 1 (120°)	Red	#e74c3c
Hat 2 (240°)	Purple	#8e44ad
Shared vertex marker	Dark red	#c0392b

The Generated Figure

The output `img/composing.svg` (converted here to PDF for inclusion) shows the three hats meeting at the shared vertex `v[12]`, namely $(3, 1, -1)$, marked by the dark-red disc:



To regenerate: run `./bin/composing` from the project root (writes `img/composing.svg`), then convert with `inkscape img/composing.svg -export-type=pdf -export-filename=manual/details/composing_figure.pdf`.

Architecture Summary

```
main()
+-- tile_load("tiles/hat.tile", &tile)
|
+-- rot[0]=base, rot[1]=t2(base) [120deg], rot[2]=t4(base) [240deg]
+-- center = tile.base.v[12] = (3,1,-1)
+-- poly[0]/hats[0] = rot[0]
|
+-- for k=1,2:                                (the attachment loop)
|   build_boundary_edges(&poly[k-1], edges) -> ec
|   for be in 0..ec, te in 0..rot[k].n:
|       try_attach_tile_poly_ex(&poly[k-1], &rot[k], .., &out, &aligned)
|       require out.cycle_count==1
|       require coord_eq(aligned.v[12], center)
|       -> hats[k]=aligned, poly[k]=out
|
+-- bounding box over hats[*].v[*] -> SC=50, M=24
+-- draw_hat x3 (amber / red / purple)
+-- centre disc + "v[12]" label
+-- write img/composing.svg
```

Usage

```
# Build (Makefile or CMake target name: composing)
make composing

# Run from the project root (relative paths to tiles/ and img/)
./bin/composing
# -> writes img/composing.svg, prints "Wrote img/composing.svg"
```

Working directory matters. `main` uses the relative paths `tiles/hat.tile` and `img/composing.svg`, so the program must be launched from the project root. In CLion, set the run configuration's *Working directory* to the project root accordingly.